

CongruentClosureDAG

Overview

This repository implements a Directed Acyclic Graph (DAG)-based congruence closure algorithm for solving the satisfiability of logical formulas in the quantifier-free fragment of equality theories. It supports multiple theories, including:

- **Equality Theory:** Manages formulas with equality and inequality relations.
- **Array Theory:** Handles array operations such as `select` and `store`.
- **List Theory:** Supports operations like `car`, `cdr`, and `cons`.

Features

- **Optimizations:** Includes forbidden sets, efficient representative selection in `UNION`, and a non-recursive `FIND` function to enhance performance.
 - **Scalability:** Built to handle complex logical formulas efficiently.
 - **Transformation Support:** Includes the ability to convert formulas between Conjunctive Normal Form (CNF) and Disjunctive Normal Form (DNF).
-

How to Run the Project

The project integrates formula parsing, congruence closure calculation, and visualization. To execute the project, follow the steps below from the `ccDag` directory:

Prerequisites

- **Java Version:** Ensure Java 11.0.21 or later is installed.
- **Dependencies:** Include the required libraries: `gs-core-2.0.jar` and `gs-ui-swing-2.0.jar`.

Compile the Source Code

1. **Compile the Source Code:**

```
javac -cp libs/gs-core-2.0.jar:libs/gs-ui-swing-2.0.jar -d bin \  
src/App.java \  
src/GraphVisualization.java \  
src/CCAlgorithm/*.java \  
src/CCAlgorithm/parser/*.java \  
src/CCAlgorithm/bean/*.java \  
src/Transformation/Selection/*.java \  
src/Transformation/DNF2/*.java
```

Run the Application

2. Run the Application:

```
java -cp bin:libs/gs-core-2.0.jar:libs/gs-ui-swing-2.0.jar App
```

Directory Structure

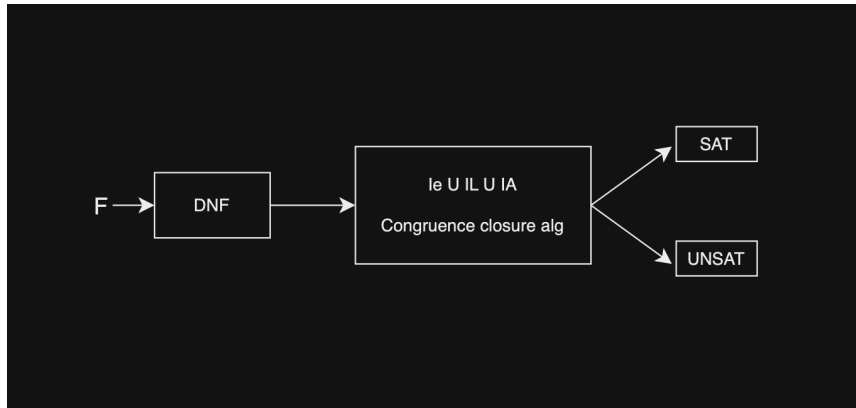
- **src/**: Contains all source code files.
 - **CCAlgorithm/**: Implements the core congruence closure algorithm.
 - **CCAlgorithm/parser/**: Includes parsers for reading logical formulas.
 - **CCAlgorithm/bean/**: Defines data structures used in the algorithm.
 - **Transformation/Selection/**: Contains selection transformation logic.
 - **Transformation/DNF2/**: Includes transformations to Disjunctive Normal Form and Conjunctive Normal Form.
 - **GraphVisualization.java**: Visualizes the congruence closure DAG.
 - **App.java**: Entry point of the application.
- **bin/**: Stores compiled `.class` files.
- **libs/**: Contains external dependencies, such as GraphStream libraries.

Notes

- Ensure the `libs` folder contains the required dependencies: `gs-core-2.0.jar` and `gs-ui-swing-2.0.jar`.
 - Before running, ensure all source files are properly compiled into the `bin` directory.
 - The project leverages Java Swing for graphical visualization of the DAG.
-

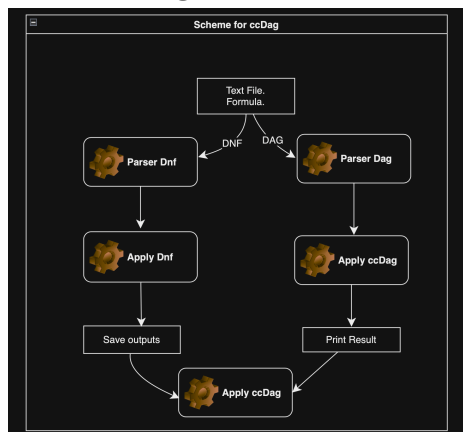
References

This implementation is inspired by Nelson-Open methods for combining decision procedures for multiple theories, with a focus on modularity and efficiency for practical use cases in SMT solving.



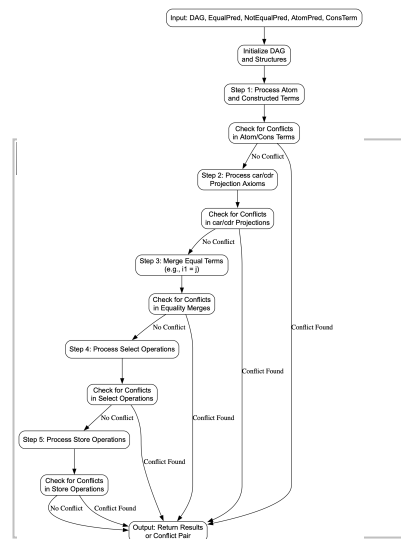
Additional Details

Full Program Schema



1. **Start:** Choose to apply DNF transformation or directly process pre-transformed formulas in the `alreadyDnfFiles/` folder.
2. **Parser:** Parses formulas to make them compatible for DAG or DNF processing.
3. **Apply DAG or DNF:** Depending on the chosen path, transform formulas into DNF or process directly using the congruence closure DAG.
4. **Output:** Save results to files for further analysis.

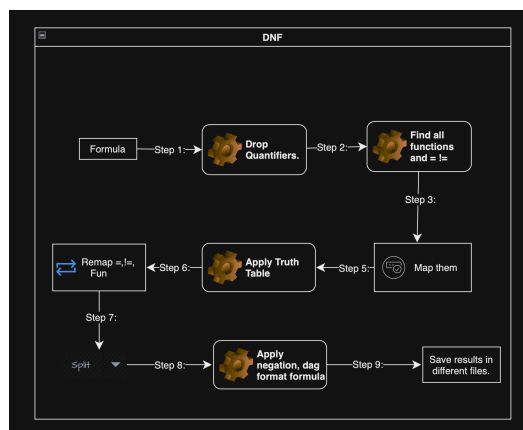
How ccDag Works:



The DAG-based congruence closure algorithm processes formulas by:

- Initializing the DAG with all terms and relationships.
- Merging terms marked as equal and checking for conflicts.
- Processing array and list operations to maintain consistency.
- Outputting the satisfiability result or the conflicting terms.

How DNF Works:



The DNF (Disjunctive Normal Form) transformation works by:

1. Dropping quantifiers like $\forall$ and $\exists$.
2. Mapping all functions and equality/inequality operations.
3. Simplifying the formula for DNF transformation.
4. Applying truth table transformations.
5. Re-mapping terms to complete the DNF transformation.
6. Saving results to files for further DAG processing.

Java Versions

This project requires Java version 11.0.21 for compatibility and performance, but it should work fine with other versions as well.

Directed Acyclic Graph (DAG)

Syntax Rules

Overview

The Directed Acyclic Graph (DAG) is used to represent dependencies and relationships between different terms and operations in a formula. It ensures efficient evaluation and transformation of logical formulas, especially when dealing with congruence closure and other complex operations in the system.

DAG Syntax Rules

- **Node Representation:** Each node in the graph represents a formula, operation, or function. Nodes are connected by directed edges that represent logical dependencies or relationships.
- **Symbols in the DAG:**
 - **Functions:** Represented by function symbols such as `Fn`, `Gn`, `Pn`, etc. Each function can have its own arity (the number of arguments it takes).
 - **Predicates:** Represented by predicate symbols such as `LEAVE`, `GO`, `OPEN`, etc. These represent logical assertions or conditions that depend on terms.
 - **Variables:** Represented by simple identifiers such as `x`, `y`, `z`, etc. These can be used as arguments for functions and predicates.
 - **Constants:** Represented by fixed values or terms like `a`, `b`, etc. These are typically used in formulas to create relationships.
- **Logical Operations:**
 - **AND (&):** The semicolon `;` in the formula represents the logical AND (&) operator, meaning that multiple conditions or expressions connected by semicolons must all hold true.
 - **Equality:** Represented as `term1 = term2`, where the terms can be variables, functions, or other sub-expressions.
 - **Inequality:** Represented as `term1 != term2`, where the terms are not equal to each other.
 - **Atoms:** Represented by `atom(term)`, where `term` can be any formula or function.
 - **Negated Atoms:** Represented as `-atom(term)`, where the negation of the atom is considered in the formula.
 - **Predicates:** Represented by `PRED(term1, term2, ...)`, where `PRED` can be any of the predicate symbols and `term1`, `term2`, etc., are the arguments.
- **Example Expressions:**

- `-COME (Gn (y, i) , Fn (i, a) ) ;`
- `-atom (Gn (e, i) ) ;`
- `FORCE (u, Hn (Rn (u) , o, Pn (y, c, a, x, e) ) , Rn (cons (o, o) ) , e, car (z) ) ;`
- `-atom (Fn (z, c) ) ;`
- `u!=b;`
- `cons (c, y) !=x;`
- `atom (Gn (car (e) , Sn (b, a, y) ) ) ;`
- `Pn (Rn (b) , a, Hn (e, u, c) , y, z)=cdr (z) ;`
- `Gn (z, i)=Pn (Sn (u, x, c) , u, cdr (z) , cdr (b) , i) ;`
- `Sn (Rn (z) , y, cons (u, i) )=cons (c, a) ;`

Important Notes

- **Functionality:** Be mindful of the operations represented in the formula and how they interact. For example, `COME`, `FORCE`, and similar predicates are applied based on the specific syntax and relations they have within the DAG.
- **Symbol Use:** The symbols in the DAG, such as `Fn`, `Gn`, and `LEAVE`, have specific meanings and should be used according to the rules defined in the syntax. Each symbol corresponds to a specific operation or function, which should be properly applied to create valid logical expressions.
- **DAG Construction:** When constructing the DAG, ensure that:
 - The formulas respect the defined syntax.
 - Each node and edge in the graph represents a valid logical relation.
 - Dependencies are correctly represented using directed edges.
- **Logical Rules:** The formula generation and parsing process will ensure that all formulas are consistent with the DAG's logical structure. Care must be taken when applying operations like negation (`-`), equality (`=`), and inequality (`!=`).

By following these syntax rules, the DAG ensures that logical formulas are evaluated and transformed correctly, making the algorithm efficient and reliable.

Conclusion

The Directed Acyclic Graph (DAG) plays a crucial role in the analysis and processing of logical formulas in the congruence closure algorithm. By adhering to the syntax rules and maintaining proper node relationships, we can ensure the efficient handling of complex logical expressions.

Disjunctive Normal Form (DNF) Generator

Overview

This project is designed to generate formulas in Disjunctive Normal Form (DNF), following strict syntax and rules to ensure consistency and correctness. The generator creates formulas and stores them in the `src/inputFiles/` directory. Below are the guidelines for writing and understanding these formulas.

Rules and Syntax

Variables

1. **Normal Variables:** A variable can represent anything, such as `car(...)`, `cdr(...)`, `Fn(...)`, etc.
2. **Negated Variables:** Negations are applied either to a single variable as `~x` or to a formula as `~(formula)`.

Equality (=)

1. **Equality of Two Variables:** Use the format `a = b`, where `a` and `b` can be any expression, e.g., `car(...)`, `cdr(...)`, `Fn(...)`, etc.
2. **Negated Equality:** Use the format `~(a = b)` when negating equality.

Operators

1. **Logical OR (|):** Combines two or more expressions inside parentheses, e.g., `(a | (b & c))`.
2. **Logical AND (&):** Combines two or more expressions inside parentheses, e.g., `(a & b)`.
3. **Logical IMPLICATION (->):** Use parentheses to clarify precedence, e.g., `(a -> b)`.

- o **Important:** When combining equality with operators, follow the equality rules above.

Restrictions

1. **Do Not Write Formulas Incorrectly:**
 - o Avoid writing expressions like `(a = (a & b))`. Instead, write simpler and clear expressions, e.g., `(a = b & b = a)`.
2. **Complex Equalities:**

- Avoid writing $a = b = c$. Instead, write each equality explicitly, e.g., $(a = b \ \& \ b = c)$.

3. IFF ($\leftrightarrow$):

- Represent only as logical equivalence using implications, e.g., $((a \rightarrow b) \ \& \ (b \rightarrow a))$.

4. Complex Functions:

- Avoid overly complex nested functions, e.g., `car(cons(d,e))` is acceptable, but avoid excessive nesting.

Formula Examples

Here are examples of correctly formatted formulas:

1. `(store(x,y) = cons(a,b) & ((car(cons(d,e)) & cdr(a)) | (cdr(a) = cdr(a) & cdr(a) = cdr(a))))`
2. `(store(x,y) = cons(a,b) & (car(cons(d,e)) & cdr(a)))`
3. `(store(x,y) = cons(a,b) & car(cons(d,e)) = cdr(cons(a,b)))`
4. `((~(Fn(x,y))) != Fn(z,w) & store(a,b) != car(cons(c,d)))`
5. `(Fn(p,q) = store(x,y) | ~(Fn(a,b) != cons(c,d)))`
6. `(Fn(p,q) = store(x,y) | ~(Fn(p,q) = store(x,y)))`

Directory Structure

- **Source Files:** The main generator code is located in the `src/` directory.
- **Generated Formulas:** Formulas are saved as `.txt` files in the `src/inputFiles/` directory.

How to Use

1. **Run the Generator:** Execute the generator program by specifying the number of formulas and maximum depth.
2. **Review the Output:** Check the `src/inputFiles/` folder for the generated `.txt` files containing formulas.
3. **Verify Compliance:** Ensure all generated formulas adhere to the syntax and rules outlined above.

Notes

- Always respect the outlined rules to maintain formula correctness.
- For any modifications to the generator, ensure the new rules are consistent with the examples provided.